

ДИСЦИПЛИНА	Технологии индустриального программирования
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическое занятие
ПРЕПОДАВАТЕЛЬ	Адышкин Сергей Сергеевич
СЕМЕСТР	2 семестр, 2025-2026 гг.

Практическое занятие №7

Написание Dockerfile и сборка контейнера

Цель занятия

Освоить контейнеризацию backend-приложения на Go с помощью Docker, научиться писать Dockerfile, собирать Docker-образ и запускать контейнеризированный сервис в воспроизводимой среде.

Задачи занятия

1. Изучить назначение контейнеризации в backend-разработке.
2. Понять различие между Docker-образом и контейнером.
3. Освоить структуру и назначение файла Dockerfile.
4. Научиться использовать multi-stage build для сборки Go-приложения.
5. Освоить настройку .dockerignore.
6. Научиться собирать Docker-образ локально.
7. Научиться запускать контейнер с пробросом портов и передачей переменных окружения.
8. Освоить базовый запуск нескольких сервисов через Docker Compose.
9. Научиться проверять работу контейнеризированного приложения.

Теоретическая часть

Зачем контейнеризировать backend-приложение

При локальной разработке очень часто возникает проблема различия окружений. У одного разработчика одна версия Go, у другого — другая; на одной машине есть нужные зависимости, на другой — нет; где-то приложение запускается, а где-то нет. Контейнеризация решает эту проблему, потому что позволяет упаковать приложение вместе с известной средой выполнения.

Для backend-разработки это особенно важно, потому что сервисы должны одинаково запускаться:

- на машине разработчика;
- на тестовом сервере;
- в CI/CD;
- на VPS;
- в Kubernetes.

Контейнеризация не заменяет саму разработку, но делает её результат воспроизводимым.

Что такое Docker-образ и контейнер

Docker-образ — это шаблон, по которому создаётся контейнер. Внутри образа описано:

- какая базовая система используется;
- какие файлы скопированы;
- какие команды были выполнены при сборке;
- что именно запускать при старте.

Контейнер – это уже запущенный экземпляр образа.

Проще говоря:

- **image** – это заготовка;
- **container** – это запущенный процесс на основе этой заготовки.

Вам важно не путать эти понятия. Сначала образ собирают, потом из него запускают контейнер.

Почему для Go удобен multi-stage build

Go-сервисы обычно компилируются в бинарный файл. Это позволяет собирать приложение в одном образе, а запускать – в другом, более лёгком.

В multi-stage build:

- на первой стадии происходит сборка бинарника;
- на второй стадии остаётся только готовый бинарный файл и минимальное окружение.

Это даёт сразу несколько преимуществ:

- итоговый образ меньше;
- внутри рантайм-образа нет лишних файлов;
- сборка выглядит чище и ближе к production-подходу.

Зачем нужен .dockerignore

При сборке Docker копирует в build context файлы проекта. Если не настроить .dockerignore, туда попадут:

- .git;
- Временные файлы;
- логи;
- локальные папки со сборками;

- мусор, не нужный внутри образа.

Это замедляет сборку и увеличивает её объём. Поэтому `.dockerignore` – обязательный элемент аккуратного проекта.

Практическая часть

Общая идея проекта

В работе используется учебный сервис tasks, который слушает HTTP-порт и отдаёт ответ на тестовый маршрут.

Если у вас уже есть сервис из предыдущих практик, можно контейнеризовать именно его. Если готового сервиса нет, можно использовать минимальный учебный вариант.

Рекомендуемая структура проекта

```
pz7-docker/  
services/  
  tasks/  
    cmd/  
      tasks/  
        main.go  
  internal/  
    go.mod  
    go.sum  
  Dockerfile  
  .dockerignore  
deploy/  
  docker-compose.yml
```

Шаг 1. Подготовка учебного сервиса

Если сервис уже существует, можно использовать его.

Если сервиса нет, создайте минимальный вариант.

Файл services/tasks/cmd/tasks/main.go:

```
package main
```

```
import (  
    "encoding/json"  
    "log"  
    "net/http"  
    "os"  
)  
  
func main() {  
    port := os.Getenv("TASKS_PORT")  
    if port == "" {  
        port = "8082"  
    }  
  
    mux := http.NewServeMux()  
  
    mux.HandleFunc("/health", func(w http.ResponseWriter, r *http.Request) {  
        w.Header().Set("Content-Type", "application/json; charset=utf-8")  
        _ = json.NewEncoder(w).Encode(map[string]string{  
            "status": "ok",  
            "service": "tasks",  
        })  
    })  
  
    addr := ":" + port  
    log.Println("tasks service started on", addr)  
  
    if err := http.ListenAndServe(addr, mux); err != nil {  
        log.Fatal(err)  
    }  
}
```

Шаг 2. Инициализация Go-модуля

Перейдите в папку сервиса и выполните:

```
cd services\tasks  
go mod init example.com/tasks  
go mod tidy
```

Шаг 3. Создание .dockerignore

Создайте файл .dockerignore:

```
.git  
  
bin  
tmp  
*.log  
.idea  
.vscode  
Dockerfile.old  
.env
```

Пояснение

Этот файл нужен для того, чтобы в контекст сборки не попадали лишние файлы. Это уменьшает размер build context и делает сборку более аккуратной.

Шаг 4. Создание Dockerfile

Создайте файл Dockerfile в папке services/tasks.

```
FROM golang:1.23 AS builder  
  
WORKDIR /app  
  
COPY go.mod go.sum ./  
RUN go mod download
```



```
COPY . .
```

```
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -o /app/bin/tasks  
./cmd/tasks
```

```
FROM alpine:3.20
```

```
WORKDIR /app
```

```
COPY --from=builder /app/bin/tasks /app/tasks
```

```
EXPOSE 8082
```

```
CMD ["/app/tasks"]
```

Пояснение к Dockerfile

На первой стадии:

- используется образ с Go;
- копируются go.mod и go.sum;
- скачиваются зависимости;
- копируется исходный код;
- собирается бинарник.

На второй стадии:

- используется минимальный образ Alpine;
- копируется только бинарный файл;
- указывается порт;
- задается команда запуска.

Именно эта схема и называется multi-stage build.

Шаг 5. Сборка Docker-образа

Находясь в папке `services/tasks`, выполните:

```
docker build -t techip-tasks:0.1 .
```

Что должно получиться

После успешной сборки Docker создаст образ с тегом `techip-tasks:0.1`.

Проверить список образов можно командой:

```
docker images
```

Шаг 6. Запуск контейнера

Запустите контейнер:

```
docker run --rm -p 8082:8082 -e TASKS_PORT=8082 techip-tasks:0.1
```

Пояснение

Здесь:

- `--rm` удаляет контейнер после остановки;
- `-p 8082:8082` пробрасывает порт;
- `-e TASKS_PORT=8082` передаёт переменную окружения

внутри контейнера.

Шаг 7. Проверка работоспособности

Откройте второе окно терминала и выполните:

```
curl http://localhost:8082/health
```

Ожидаемый ответ:

```
{"status":"ok","service":"tasks"}
```

Если ответ получен, значит контейнер собран и приложение внутри него работает корректно.

Шаг 8. Анализ различия между локальным запуском и контейнером

Теперь вы должны зафиксировать важную мысль: приложение не запускается напрямую через `go run`, а выполняется внутри контейнера.

Это означает, что:

- Среда выполнения формируется `Dockerfile`;
- запуск воспроизводим;
- сервис можно перенести на другой компьютер без ручной настройки Go-окружения.

Шаг 9. Изменение переменной окружения

Остановите контейнер и запустите его снова, но с другим значением порта внутри приложения.

Поскольку в `Dockerfile` указан `EXPOSE 8082`, а сервис по умолчанию тоже использует 8082, учебно безопаснее оставить этот же порт. Но вы должны понять сами принцип передачи конфигурации через `env`, а не жёстко вшитые значения.

Шаг 10. Подготовка Docker Compose

Создайте файл `deploy/docker-compose.yml`:

```
version: "3.9"

services:
  tasks:
    build:
      context: ../services/tasks
    container_name: tasks_service
```

```
ports:
  - "8082:8082"
environment:
  TASKS_PORT: "8082"
```

Шаг 11. Запуск через Docker Compose

Перейдите в папку deploy и выполните:

```
docker compose up -d --build
```

Проверка:

```
docker compose ps
```

Просмотр логов:

```
docker compose logs -f
```

Шаг 12. Проверка после запуска через Compose

Выполните:

```
curl http://localhost:8082/health
```

Результат должен быть тем же, что и при обычном docker run.

Шаг 13. Что важно понять по итогам практики

После выполнения работы вы должны увидеть, что Docker решает не только задачу упаковки, но и задачу стандартизации запуска.

Если образ собран правильно, то не важно, где именно запускать контейнер:

- локально;
- в CI;
- на VPS;
- в оркестраторе.

Поведение должно быть предсказуемым.

Типичные ошибки

Ошибка 1. Неправильный контекст сборки

Если команда `docker build` запускается не из той папки, Docker не найдёт нужные файлы.

Ошибка 2. Сервис внутри контейнера слушает не тот адрес

В более сложных проектах приложение должно слушать `0.0.0.0`, а не только `127.0.0.1`, иначе доступ извне контейнера может быть невозможен.

Ошибка 3. В образ попадают лишние файлы

Причина — отсутствие `.dockerignore`.

Ошибка 4. Контейнер запускается, но порт недоступен

Нужно проверить `-p` и реальный порт, который слушает приложение внутри контейнера.

Ошибка 5. Переменные окружения не применяются

Нужно убедиться, что приложение действительно читает `env` через `os.Getenv`.

Контрольные вопросы

1. Чем Docker-образ отличается от контейнера?
2. Зачем нужен Dockerfile?
3. Что такое multi-stage build?
4. Почему для Go-сервисов удобно использовать multi-stage build?
5. Зачем нужен .dockerignore?
6. Для чего используются переменные окружения в контейнере?
7. Что делает флаг -p при запуске контейнера?
8. Для чего нужен Docker Compose?
9. Почему контейнеризация упрощает деплой?
10. Почему не стоит вшивать конфигурацию и секреты прямо в образ?